

EmbeddedAD: Efficient Anomaly Detection on SoCs via Vector-Quantized Reverse Distillation

Mark Deutel^{*†}, Andreas Marchl^{*}, Axel Plinge[†], Frank Hannig^{*}, and Jürgen Teich^{*}

^{*}Friedrich-Alexander-Universität Erlangen-Nürnberg (FAU)

{mark.deutel, andreas.marchl, frank.hannig, juergen.teich}@fau.de

[†]Fraunhofer Institute for Integrated Circuits, Fraunhofer IIS

axel.plinge@iis.fraunhofer.de

Abstract—Unsupervised anomaly detection (AD) is a well established approach in applications where a sufficient number of training samples exists only for the normal class, but not for the anomalous class. However, existing AD methods are complex and therefore slow to compute on embedded system-on-chip (SoC). Based on a thorough analysis of state-of-the-art AD models, we identified reverse distillation as the most promising approach for execution on SoCs. Still, the approach suffers from computational redundancies and overparameterization. To overcome these shortcomings and enable fast and efficient execution of AD on the edge, we propose EmbeddedAD, a novel deep neural network (DNN) architecture class for unsupervised AD based on vector quantized reverse distillation (VQ-RD). EmbeddedAD introduces a new bottleneck alleviator that makes the model both robust and efficient. To further simplify the practical implementation of EmbeddedAD, we propose “double distillation”, a novel training method for reverse distillation AD models that removes the reliance on large DNN models pre-trained on ImageNet and instead allows the use of lighter architectures. Compared to the original reverse distillation approach, for EmbeddedAD we observe a 97 % reduction in memory requirements and a 7.7 times faster inference latency on a Raspberry Pi 5 SoC with almost the same AD precision.

Index Terms—Efficient AI, Anomaly Detection, System on Chips, Reverse Distillation.

I. INTRODUCTION

Visual quality inspection and anomaly detection (AD) play an important role in many industrial applications, such as quality control, predictive maintenance, and operational safety, where timely detection of deviations from normal behavior can prevent costly failures. However, there are two shortcomings of existing deep neural network (DNN) based anomaly detection models. First, many of the AD models proposed in research are both parameter- and compute-intensive, making them difficult to integrate into existing processes without major changes. Second, in many real-world scenarios, anomalies are inherently rare, making labeled anomalous data either scarce or expensive to acquire. Supervised learning approaches, which use labels and corresponding loss functions to optimize a model directly toward the objective, ultimately fall short when there is insufficient data per label category.

To address these challenges, in this work, we propose EmbeddedAD, a lightweight AD DNN architecture suitable for execution on system-on-chips (SoCs). This includes the following five contributions: First, a thorough analysis of current state-of-the-art unsupervised AD methods, including

a comparison of detection performance versus computational efficiency under resource constraints commonly found on embedded SoCs. Second, a novel AD architecture class based on vector quantized reverse distillation (VQ-RD) to maximize computational efficiency on embedded CPU-only SoCs while maintaining anomaly detection performance similar to what large GPU-accelerated AD models can achieve. Third, utilizing the error of the introduced vector quantization (VQ) as an additional anomaly signal to improve the anomaly detection precision of the original reverse distillation method. Fourth, a novel training strategy for reverse distillation AD models, which we call double distillation, that reduces the reliance on large pre-trained ImageNet DNN architectures, enabling rapid end-to-end training of anomaly detection networks for edge deployment without lengthy pre-training processes. Fifth, a thorough evaluation of our approach on the MVTec AD dataset [1], a well-established industrial anomaly detection benchmark, and a Raspberry Pi 5 SoC.

As a result, this work demonstrates that unsupervised anomaly detection can be redesigned around efficiency-first principles through lightweight architectures, multi-signal detection mechanisms, and a training framework that allows for architectural optimization. This makes our approach especially suitable for SoCs, which we demonstrate in this work on a Raspberry Pi 5. Our results show a $7.7\times$ speedup in inference latency at the cost of only a small degradation in anomaly detection performance on the MVTec AD dataset (-7% AUROC and AUPRO).

II. DISCUSSION OF ANOMALY DETECTION METHODS

While reconstruction-based approaches, i.e., autoencoders, are the basic architecture for unsupervised AD [2], they have been largely superseded by embedding-based or feature similarity-based methods in terms of anomaly detection accuracy. In the following, we discuss and benchmark the suitability of these modern AD models for SoC deployment.

A. Embedding-based Methods

This category uses model distribution statistics.

1) *Patch Distribution Modeling (PaDiM)* [3]: This method combines pre-trained feature extraction with multivariate Gaussian modeling. It assumes that each spatial location in a feature map follows a predictable statistical distribution across

normal samples. Using the ResNet-18 backbone, PaDiM extracts features from three specific hierarchical levels (Level 1, 2, and 3) to capture both fine-grained texture and global semantic context. These multi-scale features are upsampled to the highest spatial resolution and concatenated. To mitigate the curse of dimensionality and reduce computational complexity, PaDiM applies random dimensionality reduction, arbitrarily subsampling indices from the channel dimension.

PaDiM is advantageous for edge deployment in the sense that it does not require DNN training (only statistical estimation) and inference is a simple matrix operation (linear complexity). However, a significant bottleneck is the memory requirement. Storage of a covariance matrix $\Sigma_{ij} \in \mathbb{R}^{C \times C}$ for each spatial location results in a memory complexity of $\mathcal{O}(W \times H \times C^2)$, where W is the width, H is the height, and C is the number of channels of a layer's feature map.

2) *PatchCore* [4]: The approach shifts from statistical estimation to a stored memory of “normal” features, assuming that anomalies are best detected by their distance to the nearest normal patch. The method extracts features from intermediate mid-level blocks (typically 2 and 3 of the ResNet18 architecture). A key novelty of the method is the computation of “locally aware patch features”: the channel-wise vectors of the feature maps are aggregated over a local spatial neighborhood (typically 3×3 pixels) using adaptive average pooling. This allows each feature vector to retain sufficient spatial context without being overly constrained to absolute positions like PaDiM. These patch features are aggregated into a memory bank.

PatchCore is widely considered the state of the art for unsupervised anomaly detection, with recent adaptations achieving significant performance improvements even in the fully unsupervised setting. However, the method has a high inference time and a high overall memory footprint. The nearest neighbor search is computationally expensive and slow compared to the matrix multiplication of e.g., PaDiM. Furthermore, the search time scales with the size of the memory bank, which often prevents real-time applications. Moreover, the size of the memory bank depends directly on (a) the size of the training set, (b) the spatial dimensions of each sample, and (c) the chosen local patch neighborhood size. As a result, memory requirements can quickly exceed the limits of common embedded SoCs.

B. Feature Similarity-based Methods

These methods exploit the representational discrepancy between a teacher and a student network.

1) *EfficientAD* [5]: The method is designed to minimize inference latency on massively parallelized GPUs, while addressing the detection of both local structural and global logical anomalies. To prevent the student from overgeneralizing and mimicking the teacher on anomalies, two specific penalties are introduced. First, a “hard feature loss” restricts the training signal (gradients) to only the pixels with the highest error (e.g., the top 0.1 %), forcing the student to focus on difficult normal features. Second, an “ImageNet penalty” which feeds random

out-of-distribution images to the student and penalizes it if it successfully imitates the teacher.

EfficientAD achieves millisecond inference times on GPUs by eliminating memory banks, complex post-processing steps, and using custom lightweight architectures. However, on CPUs, its efficiency gains are diminished because the architecture requires the execution of three different networks (teacher, student, and autoencoder) for each inference pass. Without parallelization, this triple-pass overhead introduces a significant latency penalty compared to simpler single-pass architectures.

2) *Reverse Distillation for Anomaly Detection (RD4AD)* [6]: The approach addresses the student overgeneralization problem often found in knowledge distillation by introducing a tight information bottleneck and an inverted architectural data flow. The method consists of a fixed, pre-trained teacher encoder (e.g., ResNet18) and a trainable student decoder. Unlike standard distillation, RD4AD uses a one-class bottleneck embedding (OCBE) block in the latent space, which consists of two steps. First, a multi-scale feature fusion (MFF) block aggregates low-level and high-level features from the teacher by downsampling them to a common resolution and concatenating them. Second, a one-class embedding (OCE) block processes these fused features through multiple residue blocks, acting as an information bottleneck that removes anomalous features. Finally, the student decoder takes this compressed representation and attempts to reconstruct the original multi-scale feature maps of the teacher.

RD4AD offers a compelling trade-off for embedded SoC. By eliminating the need for nearest-neighbor search, i.e., PatchCore, or covariance matrix storage, i.e., PaDiM, it has low memory requirements. In addition, unlike EfficientAD's three-network architecture, RD4AD requires only a single forward pass, resulting in lower inference latency compared to other approaches, even on non-massively parallelized systems.

C. Benchmark on an Embedded SoC

In the following, we evaluate the four introduced state-of-the-art AD methods on the MVTec AD benchmark and a Raspberry Pi 5. Modern feature-based AD approaches saturate the benchmark, achieving high anomaly detection accuracies that consistently exceed 90 % (see Fig. 1). Among the evaluated approaches, EfficientAD exhibits the highest inference latency, requiring over 2.5 seconds per image. This massive delay is attributed to the sequential execution of its teacher, student, and autoencoder networks, which creates a bottleneck when GPU parallelization is not available.

A critical trade-off between speed and accuracy becomes apparent when analyzing the remaining models in Fig. 1. PaDiM offers the lowest inference latency, executing in less than 200 ms, but lags behind the other models in terms of detection accuracy. Conversely, PatchCore provides high accuracy but suffers from relatively high inference latency due to the computational cost of nearest-neighbor search. Within these two models, RD4AD positions itself as the most

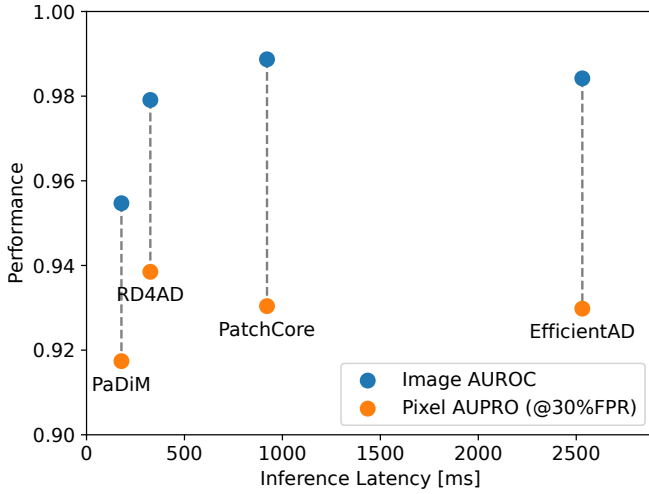


Fig. 1. Image and pixel level performance on MVtec AD vs. inference latency for the discussed state-of-the-art AD models. On the MVtec AD benchmark most SOTA models can achieve high accuracy (image AUROC >0.95 , pixel AUPRO >0.90), reaching a saturation point where the primary differentiator between models becomes their inference latency.

balanced architecture. It achieves accuracy comparable to the much slower PatchCore and EfficientAD models, while maintaining an inference speed close to the lightweight PaDiM. This suggests that the reverse distillation bottleneck efficiently captures semantic anomalies without the overhead of extensive memory searches or triple network passes.

III. LIGHTWEIGHT VECTOR-QUANTIZED REVERSE DISTILLATION

Based on the analysis in Section II, we identified reverse distillation as the most promising method for resource-constrained anomaly detection. However, the standard RD4AD implementation is both parameter- and computationally-heavy, relying on a ResNet18 backbone, a general-purpose architecture that, while accurate, was not designed with efficiency in mind. To systematically address the computational redundancies and overparameterization in RD4AD, we introduce the EmbeddedAD architecture class, see Fig. 2 for a schematic overview.

Our proposed method aims to maximize the trade-off between detection capability and computational cost through three interventions: First, the adoption of a parameter-efficient MobileNetV4 backbone, second, the redesign of the latent bottleneck block into a vector-quantized progressive feature fusion (VQ-PFF) module, which we describe in more detail below, and third, a double distillation training method to achieve full architectural flexibility without having to rely on large architectures pre-trained on ImageNet.

A. Integration of MobileNetV4

The primary computational overhead in the reverse distillation student-teacher approach stems from the extraction

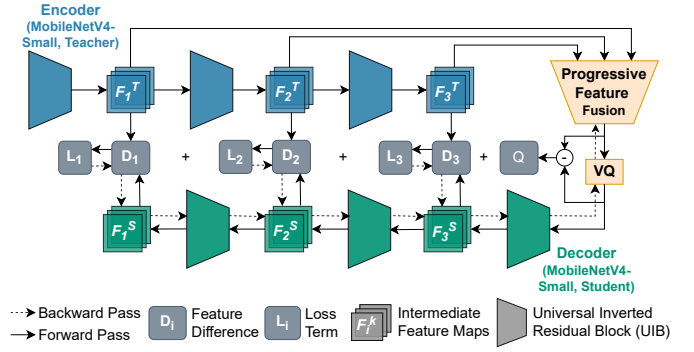


Fig. 2. Schematic overview of the proposed EmbeddedAD architecture class.

of multi-scale features. While the baseline RD4AD utilizes ResNet18, recent advances in hardware-aware architecture design, such as the MobileNetV4 family [7], have resulted in models with significantly improved computational-efficiency trade-offs. Consequently, the VQ-RD architecture adopts MobileNetV4-Small for both the teacher and student encoders.

The MobileNetV4 family utilizes universal inverted bottleneck (UIB) blocks. As a result, by integrating this backbone, the system moves away from standard dense convolutions towards depth-wise separable convolutions to maximize parameter efficiency. To maintain the semantic depth required for the student-teacher-based anomaly detection framework, features are extracted from hierarchical stages equivalent to those used in the ResNet18 baseline. Specifically, the intermediate feature maps are extracted from the outputs of stages one, two, and three of the MobileNetV4-Small architecture. This ensures that the model retains access to both the low-level texture information required for surface defect detection and the high-level structural semantics needed for logical anomalies.

B. Vector-Quantized Progressive Feature Fusion

While the introduction of the MobileNetV4 backbone reduces the cost of feature extraction, the OCBE used by the original RD4AD architecture remains computationally expensive. The standard OCBE uses dense convolution operations and brute-force channel concatenation, which creates a memory bottleneck in the latent space. To align the bottleneck with the efficiency principles of the new MobileNetV4 backbone, we propose replacing the OCBE with a VQ-PFF block (marked in orange in Fig. 2).

1) *Progressive Pairwise Feature Fusion*: Instead of aggressively downsampling with dense convolutions and aggregating multi-scale features in a single concatenation step, our proposed method fuses features iteratively. Features are fused from largest to smallest in a cascading fashion, using UIB blocks to downsample efficiently and fuse adjacent feature pairs.

This approach ensures that the efficient depth-wise separable operations are used throughout the network. In addition, the progressive approach forces the network to align low-level

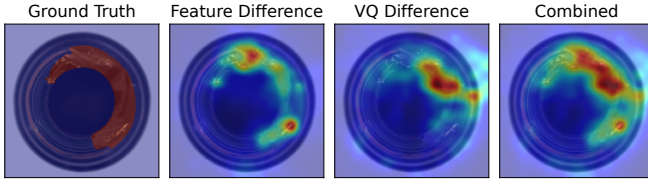


Fig. 3. An anomalous image overlayed with: (1) ground truth (red means anomalous), (2) the feature difference between student and teacher, (3) the proposed quantization error map, and (4) the final combined anomaly map.

and high-level features gradually. This reduces the maximum channel depth required at any point in the network, thereby reducing the peak memory requirement during the forward pass.

2) *Enforcing Normality via Vector Quantization*: To improve robustness without increasing model depth, a VQ mechanism as described by Van den Oord et al. [8] is integrated at the end of the feature fusion block. This layer discretizes the continuous latent space into a finite codebook of learnable prototypes. This integration serves two distinct purposes: it acts as a strong regularizer that enforces the learning of normal prototypes, and it provides an auxiliary anomaly signal based on quantization error.

The codebook is updated only with normal data during training. Consequently, the VQ layer acts as a strictly normal information bottleneck, learning prototype vectors corresponding to normal features. If, during inference, the teacher network extracts features corresponding to an anomaly, the VQ mechanism will quantize these anomalous features to the nearest existing prototype in the normal codebook. While the original OCBE block only implicitly acts as a filter for anomalous features, this mechanism directly forces the student decoder to attempt to reconstruct the teacher’s anomalous features from a normal version. This greatly increases the potential mismatch between the teacher’s output (which retains the anomaly) and the student’s output (which has been forced to look normal).

In addition to its role as a regularizer, the VQ mechanism is also used to create the quantization error Q , a secondary and explicit anomaly signal besides feature difference. We define Q as the Euclidean distance between the continuous encoder output (or, in this case, the fused version of the fusion module) and its nearest quantized neighbor in the codebook. Since the codebook approximates the manifold of normal data seen during training, the quantization error is naturally small for normal samples. Conversely, anomalous regions yield feature vectors far from the learned manifold, leading to a high quantization error. As a result, Q acts as a structural difference signal that complements the semantic feature difference signals D_1 , D_2 , and D_3 .

To combine D_1 , D_2 , and D_3 with the quantization error map Q , a normalization strategy is required that scales the maps into similar meaningful value regions, as they possess different dynamic ranges. For the scope of this work, a validation-based “MinMax” normalization is used:

- 1) *Calibration*: After model training, a validation set consisting of normal samples is utilized to track the minimum and maximum values for both the feature distance maps and the quantization error map.
- 2) *Normalization*: During inference, all maps are normalized using the tracked statistics to map them to a unified $[0, 1]$ interval. Consequently, for all maps, a value closer to 1 now means a predicted anomaly.
- 3) *Aggregation*: The final combined anomaly map is computed as the element-wise sum of the normalized feature difference and quantization error maps.

Fig. 3 visually validates this process. As shown, the quantization error map (VQ Difference) effectively isolates the anomalous regions, providing a refined localization signal that, when combined with the accumulated feature difference maps, yields a cleaner and more robust final anomaly detection map (compare Combined with Ground Truth).

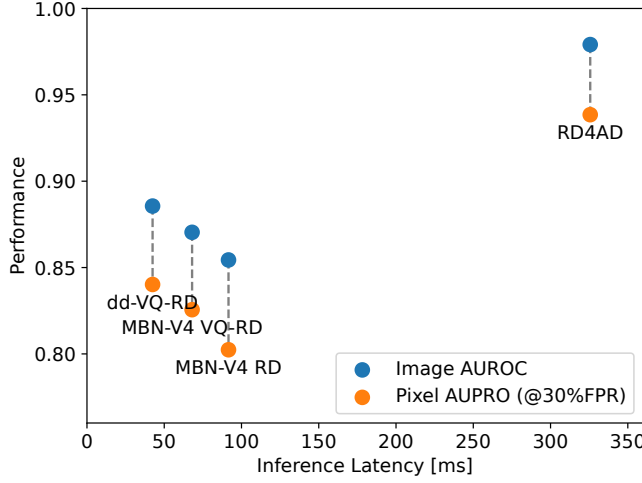
IV. THE DOUBLE DISTILLATION TRAINING METHOD

A major limitation of current unsupervised AD methods, including the standard reverse distillation framework, is the strict reliance on pre-trained backbones as encoders. This requirement limits the available design space to generic architectures, such as ResNets or standard MobileNets, for which weights trained on large datasets like ImageNet are publicly available. This dependency prevents the usage of custom, lightweight DNNs as backbones.

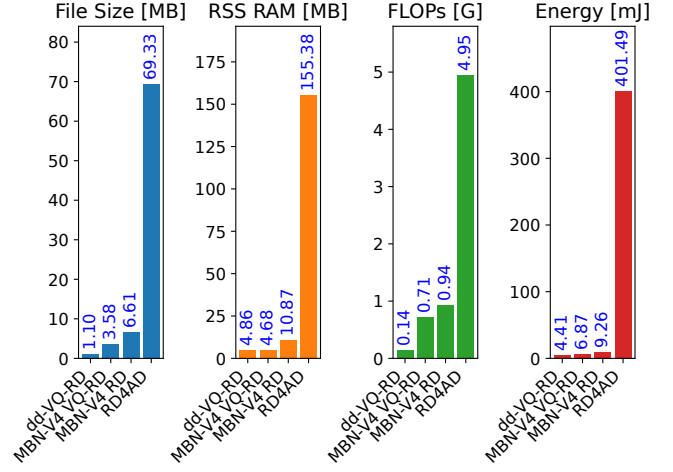
To overcome this limitation and allow full control over the model design, we propose the “double distillation” training method, based on the distillation recipe described by Bergmann et al. [5]. Our method effectively decouples AD training from the immediate availability of pre-trained weights. It allows the underlying teacher and student architectures to be optimized to minimize latency and memory requirements without sacrificing the semantic knowledge required for anomaly detection. Double distillation achieves this by splitting training into two sequential phases: an initial feature distillation phase based on knowledge distillation introduced by Hinton et al. [9] followed by standard anomaly detection training.

Double distillation begins with the encoder-to-encoder feature distillation phase, which focuses on knowledge transfer. A standard pre-trained architecture, in this case a MobileNetV4-Small, is used as a source network to provide general-purpose image features. At the same time, a custom lightweight encoder network is designed with appropriate intermediate feature levels to enable feature-based distillation. During the distillation phase, the custom network is trained to reconstruct the intermediate feature maps of the pre-trained source. By minimizing the cosine distance between the source and target feature tensors, the rich semantic representations of the larger model are transferred to the smaller custom architecture.

After the small custom encoder has learned to approximate the features of the large source model, the second phase of double distillation begins. The custom encoder is frozen and becomes the new teacher encoder network in the usual reverse distillation-based training, where a student decoder



(a) Image and pixel level performance vs. inference latency.



(b) Resource requirements.

Fig. 4. Performance on MVTec AD vs. inference latency (left) and resource requirements on Raspberry Pi 5 using onnxruntime (right) for the three proposed optimized reverse distillation models with replaced backbone (MBN-V4 RD), added latent block (MBN-V4 VQ-RD), and hyperparameter optimized backbone trained with double distillation (dd-VQ-RD) compared to the RD4AD baseline.

with an architecture identical to the teacher encoder is trained to reconstruct the features of the teacher. As a result, by using double distillation, reverse distillation training is not limited to using only large pre-trained standard backbone DNNs as teachers, but can use any neural topology without requiring any ImageNet pre-training from scratch.

V. EVALUATION

To quantify the impact of the proposed architectural changes to RD4AD, we conducted a study on the MVTec AD dataset [1]. This analysis evaluates the performance of the following three optimized architectures against the RD4AD baseline [6]:

- 1) MBN-V4 RD: RD4AD adapted to use a standard pre-trained MobileNetV4 [7] backbone (see Section III-A).
- 2) MBN-V4 VQ-RD: Like MBN-V4 RD but additionally augmented with the proposed vector-quantized bottleneck (see Section III-B).
- 3) dd-VQ-RD: MBN-V4 VQ-RD but using a custom lightweight version of MobileNetV4 pre-trained using the double distillation method instead of the regular pre-trained MobileNetV4 (see Section IV).

For evaluation, we consider two metrics: area under the receiver operating characteristic curve (AUROC) and area under the per-region overlap curve (AUPRO) [1]. AUROC is a standard metric for AD and measures the ability to discriminate between two classes (e.g., normal and abnormal images) in a binary classification setting. The value is defined as the area under the curve between the true positive rate and the false positive rate for different decision thresholds.

To evaluate pixel-level anomaly localization, we use AUPRO instead of AUROC. The reason is that AUROC would treat each pixel as an independent binary decision. Since anomalies are spatially sparse, often covering less than

1 % of an image, normal background pixels overwhelmingly outnumber anomalous ones, suppressing the false positive rate. As a result, pixel-level AUROC scores are artificially high and often mask poor localization quality. AUPRO addresses this problem by measuring the overlap between the predicted anomalous regions and the actual annotated anomalous regions in the ground truth, rather than comparing pixel by pixel.

To compare resource requirements and inference latency, we consider a Raspberry Pi 5 as an exemplary SoC. To execute the different AD models, we used onnxruntime in our experiments.

A. Performance vs. Latency Trade-off

The trade-off between detection capability and inference latency on the Raspberry Pi 5 is illustrated in Fig. 4a, where we show the performance for both averaged image-level detection (Image AUROC) and pixel-level localization (Pixel AUPRO) of all categories of the MVTec AD benchmark.

The baseline RD4AD model represents the upper bound of performance, achieving an image-level AUROC of 0.95 and a pixel-level AUPRO of 0.92. However, this accuracy comes at the cost of significant computational overhead, with inference latency exceeding 300 ms per image.

The introduction of the MobileNetV4 backbone (MBN-V4 RD) demonstrates the immediate latency benefit of using a lightweight backbone. However, while the latency is significantly reduced to about 90 ms, the detection accuracy also drops sharply to an image-level AUROC of about 0.86 and a pixel-level AUPRO of 0.80. This suggests that the reduced parameter count of the standard MobileNet encoder does not capture the full semantic richness required to distinguish subtle anomalies compared to the deeper ResNet architecture.

The introduction of the proposed VQ-PFF bottleneck (MBN-V4 VQ-RD) breaks this trend. It improves the image-level AUROC score to 0.87 while further reducing the infer-

TABLE I
IMAGE AUROC AND PIXEL AUPRO (@30%FPR) OF DD-VQ-RD (OURS)
VS. RD4AD (BASELINE) ON THE 15 CATEGORIES OF MVTec AD.

Category	dd-VQ-RD		RD4AD	
	AUROC	AUPRO	AUROC	AUPRO
Bottle	1.0000	0.9564	1.0000	0.9270
Cable	0.9653	0.8754	0.8938	0.8885
Capsule	0.8002	0.8303	0.9374	0.9330
Carpet	0.7597	0.6964	0.8977	0.9316
Grid	0.6721	0.5370	0.9727	0.9603
Hazelnut	0.9454	0.9554	0.9996	0.9503
Leather	0.9688	0.8866	0.9892	0.9609
Metal nut	0.9662	0.8815	0.9772	0.9361
Pill	0.7424	0.9019	0.9482	0.9641
Screw	0.8584	0.9171	0.9165	0.9709
Tile	0.8604	0.6531	0.9885	0.8013
Toothbrush	0.9130	0.9312	0.8444	0.9211
Transistor	0.9790	0.9175	0.9612	0.7993
Wood	0.9651	0.8430	0.9841	0.8914
Zipper	0.9310	0.8146	0.9266	0.9503
Average	0.8885	0.8398	0.9491	0.9191

ence latency to 68 ms. This confirms that the vector quantization layer acts as a resource-efficient regularizer, amplifying the anomaly signal to compensate for the lighter backbone.

The most significant improvement in efficiency is achieved by the model with the custom-optimized MobileNetV4 (dd-VQ-RD). By using double distillation for fine-tuning, the model has the fastest inference speed of approximately 42 ms, while also outperforming the standard MobileNetV4 VQ-RD model, reaching an image-level AUROC of 0.89 and a pixel-level AUPRO of 0.84. This result suggests that the feature distillation process enables the custom student to learn a more compact, task-specific representation than directly using the generic pre-trained weights. In summary, while there remains a small performance gap in both AUROC and AUPRO over the RD4AD baseline, the $7.7\times$ speedup of dd-VQ-RD compared to RD4AD on the Raspberry Pi 5 demonstrates a significant improvement in inference latency.

Finally, Table I shows the separate image-level detection and pixel-level localization scores for the 15 categories of the MVTec AD dataset. The averaged values in the last row correspond to those shown in Fig. 4a. The results in Table I show that the on average 7% decrease in AUROC and AUPRO of dd-VQ-RD compared to RD4AD is not due to an overall decrease in anomaly detection performance of dd-VQ-RD across all categories, but rather, due to it underperforming on a small subset of categories compared to the RD4AD baseline. (See the “Grid”, “Pill”, and “Carpet” categories).

B. Resource Analysis

The resource requirements of the optimized reverse distillation models are shown in Fig. 4b. It can be seen that the baseline RD4AD architecture requires nearly 5 GFLOPs per inference. In contrast, the architectural optimizations result in an order-of-magnitude reduction. The MBN-V4 RD reduces this to just under 1 GFLOPs. The final custom dd-VQ-RD model further compresses this to about 0.14 GFLOPs. This

significant reduction in FLOPs also has a direct impact on the energy consumption of the Raspberry Pi 5 per sample. The RD4AD architecture consumes around 400 mJ per sample while the final dd-VQ-RD model only consumes 4.41 mJ. We measured all energy values externally using a Joulescope DC energy analyzer.

Fig. 4b also illustrates the storage, i.e., ONNX file size, and RAM requirements of the proposed reverse distillation models on the Raspberry Pi 5 compared to the baseline RD4AD model. RD4AD has significant memory requirements, with an ONNX file size of almost 70 MB and a peak RSS RAM usage of over 150 MB. In contrast, our best performing proposed architecture, i.e., dd-VQ-RD, dramatically minimizes these memory requirements. It requires less than 2 MB of memory for the ONNX file and operates with a peak RSS of approximately 5 MB. This represents a 97% reduction in memory compared to the baseline, confirming that the double distillation method successfully distills the essential anomaly detection logic into a minimal representation suitable for execution on embedded SoCs.

VI. CONCLUSION

This work demonstrates that high-performance anomaly detection does not inherently require server-grade hardware. By re-evaluating the student-teacher paradigm through the lens of efficiency, we developed a framework that retains the semantic capabilities of large AD models while meeting the constraints of embedded SoCs. The proposed method, VQ-RD, combined with our novel double distillation training method, illustrates that architectural optimization can yield order-of-magnitude reductions in resource consumption with only a moderate trade-off in detection sensitivity.

REFERENCES

- [1] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger, “MVTec AD - A comprehensive real-world dataset for unsupervised anomaly detection,” *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 9584–9592, 2019.
- [2] M. A. Pimentel, D. A. Clifton, L. Clifton, and L. Tarassenko, “A review of novelty detection,” *Signal Processing*, vol. 99, pp. 215–249, 2014.
- [3] T. Defard, A. Setkov, A. Loesch, and R. Audigier, “PaDiM: A patch distribution modeling framework for anomaly detection and localization,” in *ICPR Workshops*, 2020.
- [4] K. Roth, L. Pemula, J. Zepeda, B. Scholkopf, T. Brox, and P. Gehler, “Towards total recall in industrial anomaly detection,” *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 14298–14308, 2021.
- [5] K. Batzner, L. Heckler, and R. König, “EfficientAD: Accurate visual anomaly detection at millisecond-level latencies,” *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pp. 127–137, 2023.
- [6] H. Deng and X. Li, “Anomaly detection via reverse distillation from one-class embedding,” *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 9727–9736, 2022.
- [7] D. Qin, C. Lechner, M. Delakis, M. Feroni, S. Luo, F. Yang, W. Wang, C. Banbury, C. Ye, B. Akin, et al., “MobileNetV4: Universal models for the mobile ecosystem,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 78–96, 2024.
- [8] A. Van Den Oord, O. Vinyals, et al., “Neural discrete representation learning,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [9] G. Hinton, O. Vinyals, and J. Dean, “Distilling the knowledge in a neural network,” *arXiv:1503.02531*, 2015.